2AMS40 Optimal Decision Making & Reinforcement Learning
Assignment 1
Group 19

Sotirios Dimitrakoulakos (2378728) Triantafyllos Fotoglou (2153904)

1 Task 1: Markov Decision Process Formulation

We formulate the maintenance problem as a discounted-cost Markov Decision Process

$$\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle,$$

with discount factor $\gamma = 0.9$. Time is discrete, $t = 0, 1, 2, \ldots$.

1.1 State Space

There are two machines $i = 1, 2$ with degradation levels

$$X_{i,t} \in \{0, \ldots, \xi_i\}, \qquad \xi_1 = 5,\ \xi_2 = 7.$$

State 0 denotes a new machine and ξ_i denotes failure.

The engineer state is described by

$$(L_t, M_t, R_t),$$

where

$$L_t \in \{0, 1, 2\}, \quad M_t \in \{0, 1, 2\}, \quad R_t \in \{0, 1, 2\}.$$

- $L_t = 0, 1, 2$ denotes depot, machine 1, machine 2.
- $M_t = 0, 1, 2$ denotes idle, travelling, repairing.
- $R_t = 0$ if idle; if travelling, it encodes the target machine; if repairing, it is the remaining repair time.

The complete state is

$$S_t = (X_{1,t}, X_{2,t}, L_t, M_t, R_t).$$

1.2 Action Space

If the engineer is idle ($M_t = 0$),

$$\mathcal{A} = \{0, 1, 2\},$$

where

$$0 = \text{do nothing}, \quad 1 = \text{repair machine 1}, \quad 2 = \text{repair machine 2}.$$

If $M_t \neq 0$, actions are not interruptible and

$$\mathcal{A}(s) = \{0\}.$$

If a machine is failed ($X_{i,t} = \xi_i$) and the engineer is idle, action 0 is not allowed.

1.3 Transition Dynamics

Machine degradation follows

$$X_{i,t+1} = \min\{X_{i,t} + P_{i,t+1}, \xi_i\}, \quad P_{i,t} \sim \text{Poisson}(0.5),$$

independent across machines and time.

Degradation occurs:

- during do-nothing periods,
- during each travel time unit,
- on the non-repaired machine during repair.

Travel.

- Depot $\leftrightarrow$ machine: 1 time unit.
- Machine $i \to$ machine j: 2 time units (via depot).

During each travel unit, both machines degrade.

Repair. Repair duration depends on the state at initiation:

$$\text{Preventive: 1 time unit if } X_{i,t} < \xi_i,$$
$$\text{Corrective: 2 time units if } X_{i,t} = \xi_i.$$

During repair:

- the repaired machine is frozen,
- the other machine degrades.

After completion,

$$X_{i,t} = 0 \quad \text{with probability 1.}$$

1.4 Cost Structure

The one-step cost is

$$c(s,a) = c_{\text{maint}}(s,a) + c_{\text{unavail}}(s).$$

Maintenance cost:

$$c_{\text{maint}}(s,a) = \begin{cases} 1 & \text{preventive repair,} \\ 5 & \text{corrective repair,} \\ 0 & \text{otherwise.} \end{cases}$$

Unavailability cost: each failed or currently repaired machine incurs cost 1 per time unit.

Reward is

$$R(s,a) = -c(s,a).$$

1.5 Final MDP Definition

The problem is fully defined by

$$\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle.$$

2 Task 2: Bellman Optimality Equations

For the above MDP, the optimal value function satisfies

$$V^*(s) = \min_{a \in \mathcal{A}(s)} \left[c(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) V^*(s') \right].$$

The optimal policy is

$$\pi^*(s) = \arg\min_{a \in \mathcal{A}(s)} \left[c(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right].$$

Structure of the Bellman Equations

Because the system evolves deterministically in the engineer component and stochastically in machine degradation, we distinguish three structural cases.

1. Idle States ($M = 0$) If $s = (x_1, x_2, L, 0, 0)$:

- For $a = 0$, both machines degrade according to the Poisson kernel.
- For $a = i$, the engineer first travels toward machine i (possibly via depot), and degradation occurs during each travel leg.

Thus

$$V^*(s) = \min_{a \in \mathcal{A}(s)} \left[c(s, a) + \gamma \sum_{d_1, d_2} p(d_1)p(d_2)V^*(s') \right],$$

where s' encodes the appropriate next mode (idle, travelling, or repairing).

2. Travelling States ($M = 1$) If $s = (x_1, x_2, 0, 1, R)$:

- Travel lasts exactly one time unit.
- Both machines degrade.
- The next state is a repair state at the target machine.

$$V^*(s) = c(s, 0) + \gamma \sum_{d_1, d_2} p(d_1)p(d_2)V^*(x_1', x_2', L', 2, R'),$$

where L' is the arrival machine and R' the repair duration.

3. Repair States ($M = 2$) If $s = (x_1, x_2, i, 2, R)$:

- Machine i is frozen.
- The other machine degrades.
- If $R = 1$, repair completes and $X_i = 0$.
- Otherwise R decreases by 1.

$$V^*(s) = c(s, 0) + \gamma \sum_{d} p(d)V^*(s'),$$

where s' reflects the deterministic repair countdown and reset.

These equations, together with the transition structure defined in Task 1, fully characterize the optimal value function V^*.

3 Task 3: Evaluation of the Corrective Only Policy

In this task we compute the total expected discounted cost of a policy that performs maintenance only upon failure. Both machines start in a healthy state and the engineer is idle at the depot:

$$s_0 = (0, 0, 0, 0, 0).$$

Policy Definition

The corrective-only policy π^{corr} is defined as follows:

- If the engineer is travelling or repairing, continue the current action.
- If idle:
  - If $X_{1,t} = \xi_1$, repair machine 1.
  - Else if $X_{2,t} = \xi_2$, repair machine 2.
  - Otherwise, do nothing.

Thus, no preventive maintenance is performed.

Exact Policy Evaluation

For a fixed policy, the value function satisfies the Bellman equation

$$V^{\pi}(s) = c(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^{\pi}(s').$$

We solve this system using value iteration (policy evaluation). The algorithm converged in 134 iterations with tolerance

$$\delta = 10^{-6}.$$

For the initial state,

$$V^{\pi^{\text{corr}}}(s_0) = 6.326107.$$

This value is exact, since it is obtained by solving the Bellman equations for the fixed policy over the full state space.

Policy Evaluation Algorithm (Pseudocode)

```
Initialize V(s) = 0 for all states
repeat
    for each state s:
        a = _corr(s)
        V_new(s) = c(s,a)
                   +  * sum_{s'} P(s'|s,a) V(s')
    compute max_diff = max_s |V_new(s) - V(s)|
    V ← V_new
until max_diff <
return V
```

Monte Carlo Verification

To validate the analytical result, we perform Monte Carlo simulation.

- Number of simulations: $n = 10{,}000$,
- Horizon length: 500,
- Discount factor: $\gamma = 0.9$.

The truncation horizon is sufficient since

$$\gamma^{500} \approx 1.32 \times 10^{-23},$$

so further discounted contributions are negligible.

The Monte Carlo estimate of the expected discounted cost is

$$\widehat{V}(s_0) = 6.302513,$$

with standard error

$$\mathrm{SE} = 0.020606,$$

and 95% confidence interval

$$[6.262126,\ 6.342900].$$

The exact value 6.326107 lies inside this interval, confirming correctness of the implementation.

The number of simulations was chosen such that the standard error falls below 0.03, ensuring sufficient statistical precision.

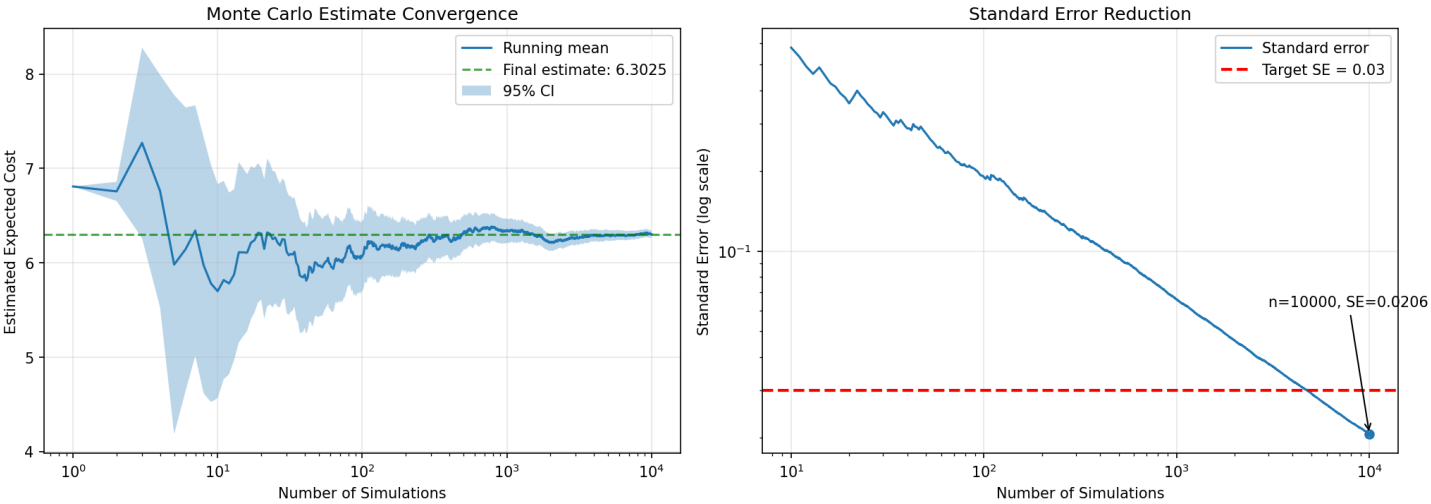

Figure 1: Monte Carlo convergence of the discounted cost estimate (running mean) and decay of the standard error as the number of simulations increases.

Discussion

The corrective-only policy yields a relatively high cost because:

- Failures trigger corrective maintenance cost 5.
- Corrective repair lasts two time units.
- Failed machines generate unavailability cost during repair.

Waiting until failure therefore leads to significant accumulated discounted cost, which motivates the computation of an optimal preventive strategy in Task 4.

4 Task 4: Optimal Policy via Value Iteration

The aim of this task is to compute the optimal policy and the corresponding total expected discounted cost per state.

Choice of Algorithm

We choose *value iteration* instead of policy iteration.

- The Bellman optimality equations are non-linear and have no closed-form solution.
- The state space is finite (432 states) and the discount factor satisfies $\gamma = 0.9 < 1$, so value iteration is guaranteed to converge.
- Value iteration only requires repeated applications of the Bellman backup and does not require solving large linear systems as in policy iteration.
- We already precomputed the transition kernel $P(s'|s,a)$ in Task 1, which allows efficient Bellman updates.

Value iteration is therefore simple to implement, numerically stable, and well suited for this problem.

Bellman Optimality Equation

For each state s the optimal value function satisfies

$$V^*(s) = \min_{a \in \mathcal{A}(s)} \left[c(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right].$$

An optimal policy is then given by

$$\pi^*(s) = \arg\min_{a \in \mathcal{A}(s)} \left[c(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right].$$

Pseudocode

```
Algorithm: Value Iteration

Input: state space S, action sets A(s), costs c(s,a),
       transition probabilities P(s'|s,a),
       discount factor , tolerance .

Initialise V_0(s) = 0 for all s in S
k = 0

repeat
    for each state s in S:
        V_{k+1}(s) = min_{a in A(s)}
                        [ c(s,a) +
                           * sum_{s'} P(s'|s,a) V_k(s') ]
    compute max_diff = max_s |V_{k+1}(s) - V_k(s)|
    k = k + 1
until max_diff <

For each state s, define
    *(s) = argmin_{a in A(s)}
                [ c(s,a) +
                   * sum_{s'} P(s'|s,a) V_k(s') ]

Output: V*(s) = V_k(s), optimal policy *(s)
```

This pseudocode can be implemented directly using the MDP components from Task 1.

Convergence Criterion and Illustration

Convergence is measured in the max-norm:

$$\Delta_k = \max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)|.$$

We stop as soon as

$$\Delta_k < \delta, \qquad \delta = 10^{-6}.$$

Figure 2 shows Δ_k on a logarithmic scale for each iteration. Value iteration converged in 126 iterations, with the last values

$$\Delta_{122} = 1.47 \cdot 10^{-6},\ \Delta_{123} = 1.33 \cdot 10^{-6},\ \Delta_{124} = 1.19 \cdot 10^{-6},\ \Delta_{125} = 1.07 \cdot 10^{-6},\ \Delta_{126} = 9.67 \cdot 10^{-7}.$$

Because $\gamma < 1$ and the Bellman operator is a contraction, the limit V^* is unique.

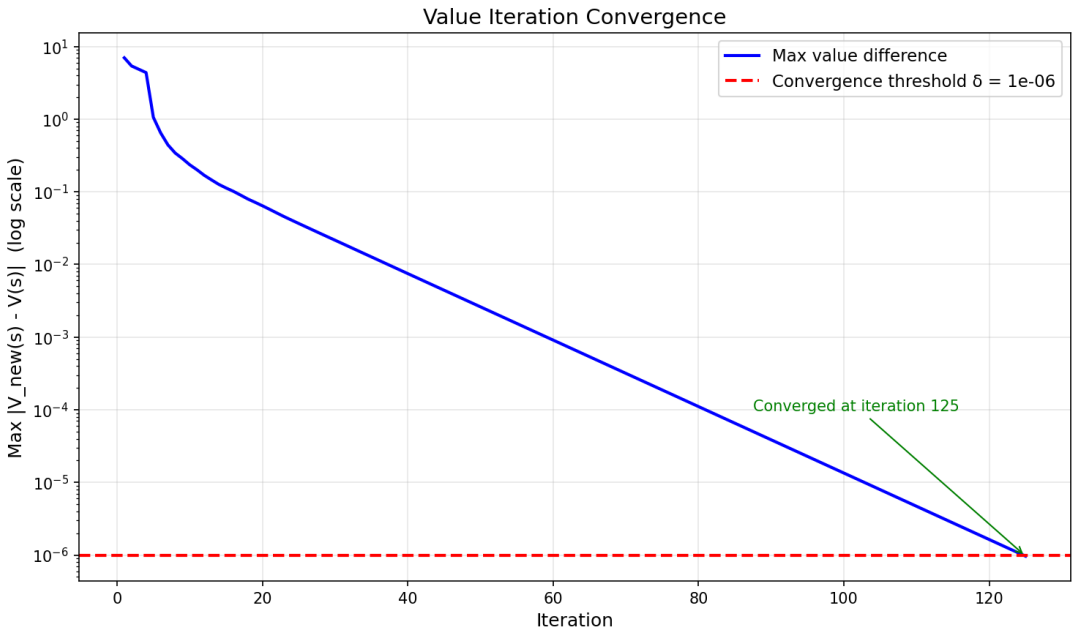

Figure 2: Value iteration convergence: max-norm difference Δ_k vs. iteration.

Optimal Cost for the Initial State

For the initial state

$$s_0 = (0, 0, 0, 0, 0)$$

(both machines healthy, engineer idle at depot), the algorithm yields the optimal expected discounted cost

$$V^*(s_0) = 3.219846.$$

Comparing with the corrective-only policy of Task 3, which had cost 6.326107, the optimal policy reduces cost by approximately 49.1%.

Optimal Policy: "Do Nothing" vs "Do Maintenance"

We now document, for the most relevant decision states, whether the optimal action is "do nothing" or "do maintenance".

When the engineer is idle at the depot ($L = 0$, $M = 0$, $R = 0$), the optimal action depends on the degradation levels of the machines. Table 1 shows the optimal action for all depot states, where

$$\text{N} = \text{“do nothing”}, \quad \text{M1} = \text{“repair machine 1”}, \quad \text{M2} = \text{“repair machine 2”}.$$

Table 1: Optimal action at the depot for each (x_1, x_2) pair.

	$x_2 = 0$	1	2	3	4	5	6	7
$x_1 = 0$	N	N	N	N	N	M2	M2	M2
1	N	N	N	N	N	M2	M2	M2
2	N	N	N	N	M2	M2	M2	M2
3	M1	M1	M1	M2	M2	M2	M2	M2
4	M1	M1	M1	M1	M1	M2	M2	M2
5	M1	M1	M1	M1	M1	M1	M1	M2

The structure is intuitive:

- For low degradation levels ($x_1 \leq 2$ and $x_2 \leq 3$), it is optimal to do nothing.

- When machine 2 is highly degraded ($x_2 \geq 5$) and machine 1 is still relatively healthy, the optimal action is to repair machine 2.
- When machine 1 is strongly degraded ($x_1 \geq 3$ and x_2 small), the optimal action is to repair machine 1.

For states where the engineer is idle at a machine ($L = 1$ or 2, $M = 0$), the policy has a similar threshold structure: if the local machine is sufficiently degraded, it is repaired immediately (without travel); otherwise the decision mirrors the depot case. Due to space constraints we omit the full tables, but they can be reconstructed directly from our implementation.

Total Expected Discounted Cost per State

Value iteration computes $V^*(s)$ for *all* 432 states. Table 2 reports the optimal value for a set of representative states with the engineer idle at the depot:

Table 2: Sample optimal values $V^*(s)$ for depot states ($L = 0, M = 0, R = 0$).

State $s = (x_1, x_2, 0, 0, 0)$	Action	$V^*(s)$
$(0, 0, 0, 0, 0)$	N	3.2198
$(0, 5, 0, 0, 0)$	M2	4.8737
$(2, 4, 0, 0, 0)$	M2	4.9582
$(3, 0, 0, 0, 0)$	M1	4.5367
$(3, 3, 0, 0, 0)$	M2	5.7319
$(4, 4, 0, 0, 0)$	M1	7.3984
$(5, 7, 0, 0, 0)$	M2	17.5674

These values illustrate how the expected cost grows with degradation and how the optimal policy selects between "do nothing" and the two maintenance actions to minimise this cost. The full vector $V^*(s)$ for all states is available from the code and can be reproduced using our implementation.

Citations and References

Use of AI and Programming Tools Minor assistance from GitHub Copilot was used during the implementation phase for debugging purposes and for generating inline code comments. Moreover, Chat GPT was used for language polishing. All modelling decisions, mathematical formulations, and algorithmic implementations were designed, analysed, and verified by the authors.

All figures and plots presented in this report were generated using the `matplotlib` Python library.

Course Material The theoretical foundations of this assignment are based on the official lecture notes, exercise sessions, and course material of the course 2AMS40 – Optimal Decision Making & Reinforcement Learning (Eindhoven University of Technology).